Lab 11 – MVVM, Commands, IDataErrorInfo, Validation

Title: Full MVVM Implementation with Validation and Commands

Objectives

This final lab completed the transition to the MVVM architectural pattern. The goals were to implement `ICommand` for handling user actions instead of Click events, implement `IDataErrorInfo` for robust validation with UI feedback, and use custom converters for complex bindings like RadioButton selection. This makes the application professional, testable, and maintainable.

Implementation

Angajat Class with IDataErrorInfo

```
using System;
using System.ComponentModel;
using System.Runtime.CompilerServices;

namespace SalariiModel
{
   public class Angajat : INotifyPropertyChanged, IDataErrorInfo
   {
      // ... fields and properties ...

      // IDataErrorInfo Implementation
      public string Error => null;
```

```
public string this[string columnName]
{
   get
   {
      switch (columnName)
      {
         case nameof(Nume):
            if (string.IsNullOrWhiteSpace(Nume))
               return "Numele este obligatoriu.";
            if (Nume.Length > 50)
               return "Numele este prea lung (max 50 caractere).";
            break;
         case nameof(Prenume):
            if (string.IsNullOrWhiteSpace(Prenume))
               return "Prenumele este obligatoriu.";
            if (Prenume.Length > 50)
               return "Prenumele este prea lung (max 50 caractere).";
            break;
         case nameof(Functie):
            if (string.IsNullOrWhiteSpace(Functie))
               return "Funcția este obligatorie.";
            if (Functie.Length > 50)
               return "Funcția este prea lungă (max 50 caractere).";
            break;
         case nameof(SalariuBrut):
            if (TipSalariu == TipSalariu.Lunar && SalariuBrut < 0)
               return "Salariul brut nu poate fi negativ.";
            if (TipSalariu == TipSalariu.Lunar && SalariuBrut == 0)
               return "Salariul brut trebuie să fie mai mare decât 0.";
```

```
            break;
        case nameof(TarifOra):
            if (TipSalariu == TipSalariu.Orar && TarifOra < 0)
                return "Tariful pe oră nu poate fi negativ.";
            if (TipSalariu == TipSalariu.Orar && TarifOra == 0)
                return "Tariful pe oră trebuie să fie mai mare decât 0.";
            break;
        case nameof(OreLucrate):
            if (TipSalariu == TipSalariu.Orar && OreLucrate < 0)
                return "Orele lucrate nu pot fi negative.";
            if (TipSalariu == TipSalariu.Orar && OreLucrate == 0)
                return "Orele lucrate trebuie să fie mai mari decât 0.";
            break;
        case nameof(Bonusuri):
            if (Bonusuri < 0)
                return "Bonusurile nu pot fi negative.";
            break;
        case nameof(DataAngajarii):
            if (DataAngajarii > DateTime.Today)
                return "Data angajării nu poate fi în viitor.";
            if (DataAngajarii < new DateTime(2000, 1, 1))
                return "Data angajării prea veche (după 2000).";
            break;
    }
    return null;
  }
}

public bool EsteValid => string.IsNullOrEmpty(this[nameof(Nume)]) &&
                         string.IsNullOrEmpty(this[nameof(Prenume)]) &&
```

```
                string.IsNullOrEmpty(this[nameof(Functie)]) &&
                string.IsNullOrEmpty(this[nameof(SalariuBrut)]) &&
                string.IsNullOrEmpty(this[nameof(TarifOra)]) &&
                string.IsNullOrEmpty(this[nameof(OreLucrate)]) &&
                string.IsNullOrEmpty(this[nameof(Bonusuri)]) &&
                string.IsNullOrEmpty(this[nameof(DataAngajarii)]);
    }
}
```

Complete MainViewModel

```
using SalariiModel;
using SalariiStocare;
using SalariiUI.Commands;
using System;
using System.Collections.ObjectModel;
using System.ComponentModel;
using System.Linq;
using System.Runtime.CompilerServices;
using System.Windows.Input;

namespace SalariiUI.ViewModels
{
    public class MainViewModel : INotifyPropertyChanged
    {
        private readonly IStocareData _admin;
        private ObservableCollection<Angajat> _angajati;
        private Angajat _angajatSelectat;
        private Angajat _angajatNou;
        private string _cautareNume;
        private string _cautareFunctie;
```

```
private string _statusMessage;
private bool _isEditMode;

public ObservableCollection<Angajat> Angajati
{
   get => _angajati;
   set { _angajati = value; OnPropertyChanged(); }
}

public Angajat AngajatSelectat
{
   get => _angajatSelectat;
   set
   {
      _angajatSelectat = value;
      OnPropertyChanged();
      if (value != null)
      {
         AngajatNou = value;
         IsEditMode = true;
      }
   }
}

public Angajat AngajatNou
{
   get => _angajatNou;
   set { _angajatNou = value; OnPropertyChanged(); }
}
```

```
public string CautareNume
{
    get => _cautareNume;
    set { _cautareNume = value; OnPropertyChanged(); }
}

public string CautareFunctie
{
    get => _cautareFunctie;
    set { _cautareFunctie = value; OnPropertyChanged(); }
}

public string StatusMessage
{
    get => _statusMessage;
    set { _statusMessage = value; OnPropertyChanged(); }
}

public bool IsEditMode
{
    get => _isEditMode;
    set { _isEditMode = value; OnPropertyChanged(); }
}

// Commands
public ICommand AdaugaCommand { get; }
public ICommand SalveazaCommand { get; }
public ICommand StergeCommand { get; }
public ICommand CautaCommand { get; }
public ICommand ResetCommand { get; }
```

```
    public ICommand IesireCommand { get; }


    public MainViewModel()
    {
        _admin = StocareFactory.GetAdministratorStocare();
        Angajati = new ObservableCollection<Angajat>(_admin.GetAngajati());
        AngajatNou = new Angajat();
        StatusMessage = "Gata...";


        // Initialize Commands
        AdaugaCommand = new RelayCommand(_ => AdaugaAngajat(), _ =>
AngajatNou?.EsteValid == true);
        SalveazaCommand = new RelayCommand(_ => SalveazaAngajat(), _ =>
AngajatSelectat?.EsteValid == true);
        StergeCommand = new RelayCommand(_ => StergeAngajat(), _ => AngajatSelectat !=
null && AngajatSelectat.Id > 0);
        CautaCommand = new RelayCommand(_ => CautaAngajati());
        ResetCommand = new RelayCommand(_ => ResetForm());
        IesireCommand = new RelayCommand(_ => Application.Current.Shutdown());
    }


    private void AdaugaAngajat()
    {
        try
        {
            _admin.AddAngajat(AngajatNou);
            Angajati.Add(AngajatNou);
            AngajatSelectat = AngajatNou;
            StatusMessage = $" ✅ Angajat {AngajatNou.NumeComplet} adăugat cu succes.";
            ResetForm();
        }
```

```
    catch (Exception ex)
    {
        StatusMessage = $"❌ Eroare: {ex.Message}";
    }
}

private void SalveazaAngajat()
{
    if (AngajatSelectat == null || AngajatSelectat.Id == 0) return;

    try
    {
        _admin.ModificaAngajat(AngajatSelectat.Id, AngajatSelectat);
        RefreshList();
        StatusMessage = $"✅ Angajat {AngajatSelectat.NumeComplet} actualizat.";
    }
    catch (Exception ex)
    {
        StatusMessage = $"❌ Eroare: {ex.Message}";
    }
}

private void StergeAngajat()
{
    if (AngajatSelectat == null || AngajatSelectat.Id == 0) return;

    try
    {
        string nume = AngajatSelectat.NumeComplet;
        _admin.StergeAngajat(AngajatSelectat.Id);
```

```
        Angajati.Remove(AngajatSelectat);
        AngajatSelectat = null;
        StatusMessage = $"🗑️ Angajat {nume} șters.";
    }
    catch (Exception ex)
    {
        StatusMessage = $"❌ Eroare: {ex.Message}";
    }
}

private void CautaAngajati()
{
    var rezultate = _admin.GetAngajati().AsEnumerable();

    if (!string.IsNullOrWhiteSpace(CautareNume))
    {
        string searchTerm = CautareNume.Trim();
        rezultate = rezultate.Where(a =>
            a.Nume.Contains(searchTerm, StringComparison.OrdinalIgnoreCase) ||
            a.Prenume.Contains(searchTerm, StringComparison.OrdinalIgnoreCase)
        );
    }

    if (!string.IsNullOrWhiteSpace(CautareFunctie))
    {
        string searchTerm = CautareFunctie.Trim();
        rezultate = rezultate.Where(a =>
            a.Functie.Contains(searchTerm, StringComparison.OrdinalIgnoreCase)
        );
    }
```

```
            Angajati = new ObservableCollection<Angajat>(rezultate);
            StatusMessage = $"🔍 {Angajati.Count} rezultate găsite.";
        }

        private void RefreshList()
        {
            Angajati = new ObservableCollection<Angajat>(_admin.GetAngajati());
        }

        private void ResetForm()
        {
            AngajatNou = new Angajat();
            CautareNume = string.Empty;
            CautareFunctie = string.Empty;
            RefreshList();
            StatusMessage = "Gata...";
        }

        public event PropertyChangedEventHandler PropertyChanged;
        protected void OnPropertyChanged([CallerMemberName] string name = null)
            => PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(name));
    }
}
```

EnumToBooleanConverter

```
using System;
using System.Globalization;
using System.Windows.Data;
```

```
namespace SalariiUI.Converters
{
   public class EnumToBooleanConverter : IValueConverter
   {
      public object Convert(object value, Type targetType, object parameter, CultureInfo culture)
      {
         return value != null && value.Equals(parameter);
      }

      public object ConvertBack(object value, Type targetType, object parameter, CultureInfo culture)
      {
         if (value is bool boolValue && boolValue)
            return parameter;
         return Binding.DoNothing;
      }
   }
}
```

MainWindow.xaml with Full MVVM Implementation

```
<Window x:Class="SalariiUI.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        xmlns:enums="clr-namespace:SalariiModel;assembly=SalariiModel"
        xmlns:converters="clr-namespace:SalariiUI.Converters"
        xmlns:viewModels="clr-namespace:SalariiUI.ViewModels"
        Title="Sistem Salarii - MVVM"
       Height="700"
       Width="1100"
        WindowStartupLocation="CenterScreen">

   <Window.DataContext>
        <viewModels:MainViewModel/>
    </Window.DataContext>
```

```
    <Window.Resources>
        <converters:EnumToBooleanConverter
x:Key="EnumToBooleanConverter"/>

        <SolidColorBrush x:Key="PrimaryBrush" Color="#2C3E50"/>
        <SolidColorBrush x:Key="AccentBrush" Color="#3498DB"/>
        <SolidColorBrush x:Key="DangerBrush" Color="#E74C3C"/>
        <SolidColorBrush x:Key="SuccessBrush" Color="#27AE60"/>
        <SolidColorBrush x:Key="LightBrush" Color="#ECF0F1"/>

        <Style x:Key="MenuButton" TargetType="Button">
            <Setter Property="Background" Value="Transparent"/>
            <Setter Property="Foreground" Value="White"/>
            <Setter Property="FontSize" Value="14"/>
            <Setter Property="HorizontalAlignment" Value="Stretch"/>
            <Setter Property="Padding" Value="15,10"/>
            <Setter Property="BorderThickness" Value="0"/>
            <Setter Property="Cursor" Value="Hand"/>
            <Style.Triggers>
                <Trigger Property="IsMouseOver" Value="True">
                    <Setter Property="Background" Value="#34495E"/>
                </Trigger>
            </Style.Triggers>
        </Style>

        <Style TargetType="TextBox">
            <Setter Property="FontSize" Value="13"/>
            <Setter Property="Padding" Value="5,3"/>
            <Setter Property="Margin" Value="5,2"/>
            <Setter Property="VerticalContentAlignment" Value="Center"/>
            <Setter Property="Validation.ErrorTemplate">
                <Setter.Value>
                    <ControlTemplate>
                        <Border BorderBrush="Red" BorderThickness="2"
CornerRadius="3">
                            <AdornedElementPlaceholder/>
                        </Border>
                    </ControlTemplate>
                </Setter.Value>
            </Setter>
        </Style>

        <Style TargetType="DataGrid">
            <Setter Property="FontSize" Value="13"/>
            <Setter Property="AlternatingRowBackground" Value="#F5F5F5"/>
```

```
            <Setter Property="RowHeight" Value="30"/>
            <Setter Property="HeadersVisibility" Value="Column"/>
            <Setter Property="CanUserAddRows" Value="False"/>
            <Setter Property="CanUserDeleteRows" Value="False"/>
            <Setter Property="AutoGenerateColumns" Value="False"/>
            <Setter Property="IsReadOnly" Value="True"/>
        </Style>
    </Window.Resources>

    <Grid>
        <Grid.ColumnDefinitions>
            <ColumnDefinition Width="180"/>
            <ColumnDefinition Width="*"/>
        </Grid.ColumnDefinitions>

        <!-- Vertical Menu -->
        <StackPanel Grid.Column="0" Background="{StaticResource
PrimaryBrush}">
            <Label Content="💰 Salarii" FontSize="22" FontWeight="Bold"
                   Foreground="White" HorizontalAlignment="Center"
Margin="10,20"/>
            <Button Content="🏠 Acasă" Style="{StaticResource
MenuButton}" Command="{Binding ResetCommand}"/>
            <Button Content="➕ Adaugă" Style="{StaticResource
MenuButton}" Command="{Binding AdaugaCommand}"/>
            <Button Content="🔍 Caută" Style="{StaticResource
MenuButton}" Command="{Binding CautaCommand}"/>
            <Button Content="📋 Listă" Style="{StaticResource
MenuButton}" Command="{Binding ResetCommand}"/>
            <Separator Background="#34495E" Margin="10,10"/>
            <Button Content="❌ Ieșire" Style="{StaticResource
MenuButton}"
                    Background="{StaticResource DangerBrush}"
Command="{Binding IesireCommand}"/>
        </StackPanel>

        <!-- Main Content -->
        <Grid Grid.Column="1" Margin="15">
            <Grid.RowDefinitions>
                <RowDefinition Height="Auto"/>
                <RowDefinition Height="Auto"/>
                <RowDefinition Height="*"/>
                <RowDefinition Height="Auto"/>
                <RowDefinition Height="Auto"/>
            </Grid.RowDefinitions>
```

```
            <!-- Search Bar -->
            <Border Grid.Row="0" Background="{StaticResource LightBrush}"
Padding="10" Margin="0,0,0,10" CornernerRadius="5">
                <StackPanel Orientation="Horizontal">
                    <Label Content="Caută nume:" Margin="0,0,5,0"/>
                    <TextBox Text="{Binding CautareNume,
UpdateSourceTrigger=PropertyChanged}" Width="150" Margin="0,0,10,0"/>
                    <Label Content="Funcție:" Margin="0,0,5,0"/>
                    <TextBox Text="{Binding CautareFunctie,
UpdateSourceTrigger=PropertyChanged}" Width="150" Margin="0,0,10,0"/>
                    <Button Content="Caută" Command="{Binding
CautaCommand}"
                            Background="{StaticResource AccentBrush}"
Foreground="White" Padding="15,5"/>
                    <Button Content="Reset" Command="{Binding
ResetCommand}"
                            Background="{StaticResource PrimaryBrush}"
Foreground="}" Foreground="White" Padding="15,5White" Padding="15,5"/>
                </StackPanel>
            </Border>

           "/>
                </StackPanel>
            </Border>

            <!-- Data <!-- DataGrid -->
           Grid -->
            <DataGrid Grid.R <DataGrid Grid.Rowow="1"="1" x:Name="dg
x:Name="dgAngajati"
                    Angajati"
                     ItemsSource="{Binding ItemsSource="{Binding
Angajati}" Angajati}"

                     SelectedItem="{Binding AngajatSelectat
SelectedItem="{Binding AngajatSelectat, Mode, Mode=TwoWay}"
=TwoWay}"
                      Margin                    Margin="0,10="0,10">
                <DataGrid.Columns>
                    <DataGridTextColumn Header="ID" Binding="{Binding
Id}" Width="50"/>
                    <Data">
                <DataGrid.Columns>
                    <DataGridTextColumn Header="ID" Binding="{Binding
Id}" Width="50GridTextColumn Header"/>
                    <DataGridTextColumn Header="Nume" Binding="{Binding
```

```
Nume="Nume" Binding="{Binding Nume}" Width="}" Width="100"/>
                  100"/>
                   <DataGridTextColumn Header="Prenume" Binding
<DataGridTextColumn Header="Prenume" Binding="{="{Binding PrenumeBinding
Prenume}" Width="100}" Width="100"/>
                   <Data"/>
                   <DataGridTextColumn HeaderGridTextColumn
Header="Funcție" Binding="{Binding="Funcție" Binding="{Binding Functie}"
Functie}" Width="120 Width="120"/>
                   <Data"/>
                   <DataGridGridTextColumn Header="TextColumn
Header="Departament" Binding="{Binding DepartDepartament"
Binding="{Binding Departamentament}" Width="100"/>
                   <Data}" Width="100"/>
                   <DataGridTextColumn HeaderGridTextColumn Header="Tip"
Binding="Tip" Binding="{Binding TipSal="{Binding TipSalariu}" Widthariu}"
Width="80"/>
                   <DataGridTextColumn Header="80"/>
                   <DataGridTextColumn Header="Br="Brut" Binding="{ut"
Binding="{Binding SalariuBinding SalariuBrBrutEfectutEfectiv,
StringFormativ, StringFormat={}{={}{0:C0:C}}" Width="100"/>
                   <DataGridTextColumn}}" Width="100"/>
                   <DataGridTextColumn Header Header="Net" Binding="Net"
Binding="{Binding Salari="{Binding SalariuNet, StringFormat={}{0uNet,
StringFormat={}{0:C}}" Width:C}}" Width="="100"/>
                  100"/>
                   <DataGridTextColumn <DataGridTextColumn Header="Data
Angaj Header="Data Angajării" Bindingării" Binding="{Binding
DataAng="{Binding DataAngajarii, Stringajarii,
StringFormat=Format=dd.MM.yyyy}" Width="120dd.MM.yyyy}" Width="120"/>
               </Data"/>
               </DataGrid.ColumnsGrid.Columns>
            </DataGrid>
            </DataGrid>

            <!-- Edit>

            <!-- Edit Form -->
            <GroupBox Grid Form -->
            <GroupBox Grid.Row="2.Row="2" Header"
Header="Editare="Editare Angajat Angajat" Margin="0,"
Margin="0,10,010,0,0">
               ,0">
                <Grid Margin="5">
 <Grid Margin="5">
                   <Grid.ColumnDefinitions
```

```
<Grid.ColumnDefinitions>
                        <ColumnDefinition>
                        <ColumnDefinition Width Width="Auto"/>
                        <Column="Auto"/>
                        <ColumnDefinitionDefinition Width="* Width="*"/>
                        <ColumnDefinition Width="Auto"/>
                        <ColumnDefinition Width="Auto"/>
                        <ColumnDefinition"/>
                        <ColumnDefinition Width="*"/>
                    </Grid.ColumnDefinitions>
                    Width="*"/>
                    </Grid.ColumnDefinitions>
                    <Grid.RowDefinitions>
                        <Grid.RowDefinitions>
                        <RowDefinition Height <RowDefinition
Height="Auto="Auto"/>
                        <Row"/>
                        <RowDefinition Height="Auto"/>
                        <RowDefinition Height="Auto"/>
                        <Definition Height="AutoRowDefinition
Height="Auto"/>
                        <RowDefinition Height=""/>
                        <RowAuto"/>
                        <RowDefinition Height="Auto"/>
                        <RowDefinition Height="AutoDefinition
Height="Auto"/>
                        <RowDefinition Height="Auto"/>
                        <RowDefinition Height="Auto"/>
                        <Row"/>
                        <RowDefinition Height="Auto"/>
                        <RowDefinition Height="Auto"/>
                        <RowDefinition Height="AutoDefinition
Height="Auto"/>
                    </Grid.Row"/>
                    </Grid.RowDefinitionsDefinitions>

                    <!-- N>

                    <!-- Nume -->
                    <Label Gridume -->
                    <Label Grid.Row="0" Grid.Row="0" Grid.Column="0"
Content="Nume.Column="0" Content="N:"ume:"/>
                    <TextBox Grid.Row/>
                    <TextBox Grid.Row="0" Grid.Column="="0"
Grid.Column="1"
                            1"
```

```
                        Text="{Binding AngajatSelectat Text="{Binding
AngajatSelectat.Nume, Mode.Nume, Mode=TwoWay, Validates=TwoWay,
ValidatesOnDataErrorsOnDataErrors=True,
UpdateSourceTrigger=Property=True,
UpdateSourceTrigger=PropertyChanged}"/Changed}"/>

                    <!-- Prenume -->
                    >

                    <!-- Prenume -->
                    <Label Grid.Row <Label Grid.Row="1" Grid.Column="1"
Grid.Column="0" Content="Pren="0" Content="Prenume:"/>
                    ume:"/>
                    <TextBox Grid.R <TextBox Grid.Row="1"
Grid.Column="1ow="1" Grid.Column="1"
                            Text="{Binding Ang"
                            Text="{Binding
AngajatSelectat.PajatSelectat.Prenume, Mode=Tworenume, Mode=TwoWayWay,
ValidatesOnData, ValidatesOnDataErrors=True,
UpdateSourceTrigger=PropertyErrors=True,
UpdateSourceTrigger=PropertyChanged}"/Changed}"/>

                    <!-- Funcție -->
                    >

                    <!-- Funcție -->
                    <Label Grid.Row=" <Label Grid.Row="2" Grid.Column="2"
Grid.Column="0" Content="Funcție:"0" Content="Funcție:"/>
                    <TextBox/>
                    <TextBox Grid.Row="2" Grid.Column Grid.Row="2"
Grid.Column="1"="1"
                            Text="{Binding AngajatSelectat
                            Text="{Binding
AngajatSelectat.Functie,.Functie, Mode=TwoWay, Validates Mode=TwoWay,
ValidatesOnDataErrors=True,OnDataErrors=True,
UpdateSourceTrigger=PropertyChanged}"/
UpdateSourceTrigger=PropertyChanged}>

                    <!-- Depart"/>

                    <!-- Departament -->
                    <Label Grid.Rowament -->
                    <Label Grid.Row="3" Grid="3" Grid.Column="0"
Content="Departament.Column="0" Content="Departament:"/>
                    :"/>
                    <ComboBox Grid.Row="3" <ComboBox Grid.Row="3"
```

```
Grid.Column="1 Grid.Column="1"
                            "
                             ItemsSource ItemsSource="{Binding
Source={x:="{Binding Source={x:Type enums:Type enums:Departament}}"
                              SelectedItem="{Binding AngDepartament}}"
                              SelectedItem="{Binding
AngajatSelectat.DepartajatSelectat.Departamentament, Mode=TwoWay,
Mode=TwoWay}"/>

                    <!-- Tip Salari}"/>

                    <!-- Tip Salariu (RadioButtons with Converter)u
(RadioButtons with Converter) -->
                    <Label -->
                    <Label Grid.Row="4" Grid.Column Grid.Row="4"
Grid.Column="0" Content="0" Content="Tip Salariu:"/>
                   ="Tip Salariu:"/>
                    <StackPanel Grid.Row="4 <StackPanel Grid.Row="4"
Grid.Column="1" Orientation="" Grid.Column="1" Orientation="Horizontal">
                       Horizontal">
                        <RadioButton Content=" <RadioButton
Content="Lunar"
                                    Lunar"
                                     IsChecked="{Binding AngajatSelectat
IsChecked="{Binding AngajatSelectat.TipSalari.TipSalariu,
Converter={StaticResource Enumu, Converter={StaticResource
EnumToBooleanConverter},ToBooleanConverter}, ConverterParameter={x:Static
en ConverterParameter={x:Static
enums:ums:TipSalariu.Lunar}}"TipSalariu.Lunar}}"
                                     Margin="5"/>
                        <Radio
                                     Margin="5"/>
                        <RadioButton Content="OrButton Content="Orar"
                                     IsChecked="{ar"
                                     IsChecked="{Binding
AngajatSelectatBinding AngajatSelectat.TipSalariu,
Converter={StaticResource EnumTo.TipSalariu, Converter={StaticResource
EnumToBooleanConverter}, ConverterBooleanConverter},
ConverterParameter={x:Static enumsParameter={x:Static
enums:TipSalariu:TipSalariu.Orar}}".Orar}}"
                                     Margin="5"/>
                    </
                                     Margin="5"/>
                    </StackPanel>

                    <!--StackPanel>
```

```
                    <!-- Salariu Brut Salariu Brut -->
                    <Label Grid -->
                    <Label Grid.Row="5" Grid.Column.Row="5"
Grid.Column="0" Content="0" Content="Salariu="Salariu Brut:"/>
                    <TextBox Grid Brut:"/>
                    <TextBox Grid.Row.Row="="5" Grid.Column="1"5"
Grid.Column="1"
                             Text="{
                             Text="{Binding AngBinding
AngajatSelectat.SalariajatSelectat.SalariuBrut, Mode=TwouBrut,
Mode=TwoWay, ValidWay, ValidatesatesOnDataErrorsOnDataErrors=True,
Update=True,
UpdateSourceTrigger=PropertySourceTrigger=PropertyChangedChanged}"/>

                    <!-- Tarif Oră -->
                   }"/>

                    <!-- Tarif Or <Label Grid.Rowă -->
                    <Label Grid.Row="5" Grid="5"
Grid.Column="2".Column="2" Content="Tarif Content="Tarif Oră:" Oră:"/>
                    <TextBox Grid/>
                    <TextBox Grid.Row="5.Row="5" Grid.Column=""
Grid.Column="3"
                             Text="{Binding Ang3"
                             Text="{Binding
AngajatSelectat.TajatSelectat.TarifOraarifOra, Mode=Two, Mode=TwoWay,
ValidWay, ValidatesatesOnDataErrors=TrueOnDataErrors=True, Update,
UpdateSourceTrigger=PropertyChanged}"/>

                    <!--SourceTrigger=PropertyChanged}"/>

                    <!-- Ore Lucrate Ore Lucrate -->
                    <Label -->
                    <Label Grid.Row=" Grid.Row="6" Grid.Column6"
Grid.Column="0" Content="0" Content="="Ore Lucrate:"/>
                   Ore Lucrate:"/>
                    <TextBox Grid.R <TextBox Grid.Row="6ow="6"
Grid.Column="1" Grid.Column="1"
                             Text"
                             Text="{Binding AngajatSelectat.Ore="{Binding
AngajatSelectat.OreLucrate, ModeLucrate, Mode=TwoWay, ValidatesOn=TwoWay,
ValidatesOnDataDataErrors=True, UpdateErrors=True,
UpdateSourceTrigger=PropertyChanged}"/SourceTrigger=Property>

                    <!-- BonusuriChanged}"/>
```

```
                    <!-- Bonusuri -->
                    <Label Grid.Row -->
                    <Label Grid.Row="="6" Grid.Column6" Grid.Column="2"
Content="2" Content="Bonus="Bonusuri:"/>
                    <TextBoxuri:"/>
                    <TextBox Grid.Row=" Grid.Row="6" Grid.Column="3"6"
Grid.Column="3"
                             Text="{Binding Angajat
                             Text="{Binding
AngajatSelectSelectat.Bonusuriat.Bonusuri, Mode=Two, Mode=TwoWay,
ValidatesOnDataErrors=TrueWay, ValidatesOnDataErrors=True,
UpdateSourceTrigger=PropertyChanged}"/>

, UpdateSourceTrigger                    <!-- Data
Angajarii=PropertyChanged}"/>

                    <!-- -->
                    Data Angajarii -->
                    < <Label Grid.Row="7Label Grid.Row="7" Grid.Column=""
Grid.Column="0" Content="Data Angajă0" Content="Data Angajării:"/>
                   rii:"/>
                    <DatePicker Grid.Row="7 <DatePicker Grid.Row="7"
Grid.Column="" Grid.Column="1"
                              1"
                               SelectedDate="{Binding AngajatSelectat
SelectedDate="{Binding AngajatSelectat.DataAngajarii, Mode.DataAngajarii,
Mode==TwoWay, ValidatesTwoWay, ValidatesOnDataErrorsOnDataErrors=True}"/>

                    <!--=True}"/>

                    <!-- Buttons -->
                    Buttons -->
                    < <StackPanel Grid.Row="7" Grid.Column="2StackPanel
Grid.Row="7" Grid.Column="2" Grid.ColumnSpan="2"" Grid.ColumnSpan="2"

                               Orientation="Horizontal"
HorizontalAlignment="Right Orientation="Horizontal"
HorizontalAlignment="Right">
                        <Button">
                        <Button Content="Salvează" Command
Content="Salvează" Command="{Binding Sal="{Binding SalveazaveazaCommand}"
                               Background="{Command}"
                               Background="{StaticResource SuccessBrush}"
ForeStaticResource SuccessBrush}" Foreground="Whiteground="White"
Padding" Padding="15,8="15,8"/>
```

```
                      "/>
                       <Button Content <Button Content="="ȘterȘterge"
Command="{Binding StergeCommandge" Command="{Binding Sterge}"
                                Background="{StaticResource
DangerCommand}"
                                Background="{StaticResource DangerBrush}"
ForegroundBrush}" Foreground="White" Padding="White" Padding="15,8"/>
                       <Button="15,8"/>
                       Content="Reset" <Button Content="Reset"
Command="{Binding ResetCommand Command="{Binding ResetCommand}"
                               }"
                                Background="{StaticResource
Background="{StaticResource PrimaryBrush}" Foreground="White"
PrimaryBrush}" Foreground="White" Padding="15, Padding="15,8"/>
                    </StackPanel>
                8"/>
                    </ </Grid>
            StackPanel>
                 </Grid>
             </GroupBox </GroupBox>

             <!-- Status Bar>

             <!-- Status Bar -->
             < -->
             <StatusStatusBar Grid.RowBar Grid.Row="3" Margin="3"
Margin="0,10,0,0="0,10,0,0">
                ">
                 <StatusBarItem>
                     <StatusBarItem>
                     <TextBlock Text <TextBlock Text="{Binding
StatusMessage}="{Binding StatusMessage}"/>
                 </StatusBar"/>
                 </StatusBarItem>
Item>
                 <Separator                <Separator/>
                />
                 <StatusBarItem>
                     <StatusBarItem>
                     <TextBlock Text="{ <TextBlock Text="{Binding
AngajBinding Angajati.Count, StringFormat='Total:ati.Count,
StringFormat='Total: { {00} angajați'}"/>
                 </StatusBarItem>
             </StatusBar>
         </Grid>
} angajați'}"/>
```

```
                </StatusBarItem>
            </StatusBar>
        </Grid>
    </    </Grid>
</Window>
```

Why I Used This Approach

**ICommand for Used This Approach

ICommand for Commands: Using Commands**: Using `ICommand` (ICommand(RelayCommand)`RelayCommand`) decouples user decouples user actions from UI actions from UI events. Commands can be bound events. Commands can to buttons be bound to buttons and other controls and other controls. The. The `CanExecute` logic `CanExecute` logic automatically automatically enables/dis enables/disables controls based on theables controls based on the command command's state's state.

IDataErrorInfo: This.

IDataErrorInfo: This interface provides a standardized interface provides a standardized way to implement validation way to implement validation. The. The UI automatically uses UI automatically uses it it when when `ValidatesOnDataErrors`ValidatesOnDataErrors=True`is set=True` is set on on bindings. Validation bindings. Validation errors are errors are displayed with a red border around the control.

displayed with a red border around the control.

EnumToBooleanEnumToBooleanConverter**: This customConverter**: This custom converter allows RadioButtons converter allows RadioButtons to bind to an to bind to an enum property enum property. The converter. The converter converts the enum converts the enum value to boolean value to boolean (true if the enum (true if the enum value matches the converter value matches the converter parameter) and back.

parameter) and back.

CommandCommandManagerManager.RequerySuggested**: This.RequerySuggested**: This static static event automatically event automatically refreshes command refres'shes command's `CanCanExecutestate when the UI changesExecute` state when the UI changes, ensuring, ensuring buttons are buttons are properly properly enabled/disabled.

**x enabled/disabled.

:Staticx:Static:**: The `{ The{x:Static}x:Static}` markup markup extension extension allows allows referencing static referencing static values (values (like enum valueslike enum values) in XAML) in XAML.

**Two-Way Binding with.

Two-Way Binding with Validation Validation: `Mode**:`Mode=TwoWay, ValidatesOnDataErrors=True,=TwoWay, ValidatesOnData UpdateSourceTrigger=Errors=True, UpdateSourceTrigger=PropertyChanged`provides realPropertyChanged` provides real-time validation-time validation feedback feedback as the user types.

** as the user types.

ViewModel StateViewModel State Management Management: The View**: The ViewModel manages theModel manages the state state of the of the application (which application (which employee employee is selected, whether is selected, whether in edit mode, etc in edit mode, etc.) and exposes.) and exposes it through properties it through properties.

##.

Testing & Results

I Testing & Results tested the complete

I tested the complete MVVM implementation MVVM implementation:
1.:

1. **Validation**: **Validation**: Entering invalid Enter data (ing invalid data (emptyempty fields, negative fields, negative numbers) numbers) showed red borders with showed red borders with error messages error messages
2. **Commands**: The
3. **Commands**: The Save Save and Delete buttons were and Delete buttons were properly enabled properly enabled/disabled based on the/disabled based on the state state
4. Radio
5. **RadioButton Binding**: SelectingButton Binding: Selecting "L "Lunar" orunar" or "Orar" correctly "Orar" correctly updated the `Tip updated the`TipSalariu`Salariu` property property
6. **CR
7. **CRUD Operations**: Adding, savingUD Operations**: Adding, saving, deleting, deleting, and searching all worked, and searching all worked correctly correctly
8. Status Messages
9. **Status Messages**: The: The status bar showed status bar showed appropriate messages appropriate messages for each operation for each operation

The

The application now behaves application now behaves like like a professional desktop application with proper a professional desktop application with proper validation, validation, data binding data binding, and separation, and separation of concerns.